LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – INHERITANCE



FEMBIANA DIKA SAPUTRI

103132400026

PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Inheritance merupakan konsep dalam pemrograman berorientasi objek (OOP) yang memungkinkan suatu kelas mewarisi atribut dan method dari kelas lain. Kelas yang mewariskan disebut superclass, sedangkan kelas yang menerima pewarisan disebut subclass. Inheritance digunakan untuk mengurangi duplikasi kode dan membentuk hubungan antarkelas.

Selain inheritance, terdapat beberapa konsep yang berkaitan, yaitu:

1. Keyword super digunakan untuk mengakses konstruktor, atribut, atau method milik superclass.
2. Method overloading adalah penggunaan nama method yang sama dalam satu kelas, tetapi memiliki jumlah atau tipe parameter yang berbeda.
3. Method overriding adalah pendefinisian ulang method dari superclass pada subclass dengan nama dan parameter yang sama.
4. Polymorphism memungkinkan objek dari kelas yang berbeda diperlakukan melalui tipe referensi yang sama. Dalam Java, hal ini dapat diterapkan menggunakan array bertipe superclass yang menyimpan objek dari beberapa subclass.

Konsep-konsep tersebut membantu membuat program lebih terstruktur, mengurangi pengulangan kode, dan memudahkan pengembangan program.

B. Guided

1. guided.inheritance

A.java

```
package com.fembi.guided.inheritance;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+x+", y = "+y);
    }
}
```

Output :

-

B.java

```
package com.fembi.guided.inheritance;

public class B extends A {
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = x+y+z;

        System.out.println("Jumlah (x+y+z) = " + hasil);
    }
}
```

Output :

-

Demo.java

```
package com.fembi.guided.inheritance;

public class Demo {
    public static void main(String[] args){
        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("Subclass: ");
        subclass.jumlahXYZ();
        subclass.tampilXY();
    }
}
```

Output :

```
Superclass:
Nilai x = 3, y = 4
Subclass:
Jumlah (x+y+z) = 8
Nilai x = 1, y = 2
```

2. guided.keywordSuper

Employee.java

```
package com.fembi.guided.keywordSuper;

public class Employee {
    private String name;
    String departement;

    public Employee(String name){
        this.name = name;
    }

    public void tampilNama(){
        System.out.println("nama: "+ name);
    }
}
```

Output :

-

Manager.java

```
package com.fembi.guided.keywordSuper;

public class Manager extends Employee{
    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("departement: " + super.departement);
    }
}
```

Output :

-

Demo.java

```
package com.fembi.guided.keywordSuper;

public class Demo {
    public static void main(String[] args){
        Manager adi = new Manager("Adi", "Sukabirus");
```

```
        adi.departement = "SDM";
        adi.tampilInformasi();
    }
}
```

Output :

```
nama: Adi
alamat: Sukabirus
departement: SDM
```

3. guided.overloadingMethod

Lagu.java

```
package com.fembi.guided.overloadingMethod;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }
    void IsiParam(String param1,String param2) {
        judul = param1;
        pencipta = param2;
    }

    void tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}
```

Output :

-

Main.java

```
package com.fembi.guided.overloadingMethod;

public class Main {
    public static void main(String[] args){
        Lagu d,e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2","aku");
```

```
        d.tampilkan();
        e.tampilkan();
    }
}
```

Output :

```
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : Lagu 2, pencipta : aku
```

4. guided.overidingMethod

A.java

```
package com.fembi.guided.overidingMethod;

public class A {
    public void tampilkan() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

Output :

-

B.java

```
package com.fembi.guided.overidingMethod;

public class B extends A{
    public void tampilkan() {
        super.tampilkan();
        System.out.println("Method milik class B dipanggil...");
    }
}
```

Output :

-

Main.java

```
package com.fembi.guided.overidingMethod;

public class Main {
    public static void main(String[] args) {
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output :

```
Method milik class A dipanggil...
Method milik class B dipanggil...
```

C. Unguided

Sebuah tim *machine learning* ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis *deep learning* yang merupakan pengembangan dari model klasifikasi biasa.

Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

1. Kelas induk (model umum)
   - Menyimpan nama model (teks) dan jumlah data latih (bilangan bulat).
   - Memiliki *constructor* yang menerima kedua nilai tersebut, serta cara bagi kelas lain untuk membaca jumlah data latih.
   - Memiliki method untuk menampilkan informasi.
   - Memiliki dua method untuk menghitung biaya komputasi dengan nama yang sama (*overloading*):
     - Tanpa parameter: biaya = jumlah data latih × 2.
     - Dengan satu parameter bilangan bulat / jumlah epoch: biaya = jumlah data latih × epoch.

Jawaban :

Model.java

```
package com.fembi.unguided;

public class Model {
    private String nama;
    private int jumlahDataLatih;

    public Model(String nama, int jumlahDataLatih) {
        this.nama = nama;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilkanInformasi() {
        System.out.println("Model: " + nama
                + " | Data latih: " + jumlahDataLatih);
    }
```

```
    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

Output :

-

Penjelasan :

Kelas Model menyimpan atribut nama dan jumlahDataLatih yang bersifat private. Konstruktor digunakan untuk mengisi kedua atribut tersebut saat objek dibuat. Method getJumlahDataLatih() memungkinkan subclass membaca jumlah data latih.

Kelas ini juga memiliki dua method hitungBiaya(), yaitu tanpa parameter dan dengan satu parameter epoch. Keduanya menerapkan overloading karena nama method sama, tetapi parameternya berbeda.

2. Kelas turunan pertama (model regresi)
   - Mewarisi kelas induk dan tambahkan atribut jumlah fitur.
   - *Constructor* menerima nama, jumlah data latih, dan jumlah fitur; gunakan *constructor* kelas induk.
   - Override method: tampilkan informasi dari kelas induk.
   - Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) + (jumlah fitur × 10).
   - Tambahkan satu method baru dengan nama yang sama dengan method penghitung biaya, tetapi menerima dua parameter bilangan bulat (jumlah *epoch* dan ukuran *batch*): biaya = jumlah data latih × epoch ÷ ukuran batch. Sehingga objek kelas ini memiliki tiga versi method penghitung biaya.

Jawaban :

ModelRegresi.java

```
package com.fembi.unguided;

public class ModelRegresi extends Model{
    private int jumlahFitur;

    public ModelRegresi(String nama, int jumlahDataLatih,
                        int jumlahFitur) {
        super(nama, jumlahDataLatih);
```

```
        this.jumlahFitur = jumlahFitur;
    }

    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    public int hitungBiaya(int epoch, int batch) {
        return getJumlahDataLatih() * epoch / batch;
    }
}
```

Output :

-

Penjelasan :

Kelas ini mewarisi kelas Model menggunakan extends dan menambahkan atribut jumlahFitur. Pemanggilan super(nama, jumlahDataLatih) digunakan untuk menjalankan konstruktor kelas induk.

Method tampilkanInformasi() dan hitungBiaya() tanpa parameter di-override. Biaya komputasinya dihitung dari biaya dasar ditambah jumlah fitur dikali 10. Selain itu, terdapat method hitungBiaya(int epoch, int batch) sebagai versi ketiga yang menerapkan overloading.

3. Kelas turunan kedua (model klasifikasi)

- Mewarisi kelas induk dan menambahkan atribut jumlah kelas.
- *Constructor* menerima nama, jumlah data latih, dan jumlah kelas; gunakan *constructor* kelas induk.
- Override method: tampilkan informasi dari kelas induk.
- Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) × jumlah kelas.
- Versi penghitung biaya dengan parameter *epoch* tidak di-override.

Jawaban :

ModelKlasifikasi.java

```
package com.fembi.unguided;

public class ModelKlasifikasi extends Model{
    private int jumlahKelas;

    public ModelKlasifikasi(String nama, int jumlahDataLatih,
                            int jumlahKelas) {
        super(nama, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

Output :

-

Penjelasan :

Kelas ini mewarisi Model dan menambahkan atribut jumlahKelas. Method tampilkanInformasi() di-override untuk menampilkan jumlah kelas. Method hitungBiaya() juga di-override dengan mengalikan biaya dasar dari superclass dengan jumlah kelas.

Method hitungBiaya(int epoch) tidak di-override, sehingga perhitungan biaya berdasarkan epoch tetap menggunakan method dari kelas Model.

4. Kelas turunan ketiga (model klasifikasi *deep learning*)
   Buatlah class Main untuk merangkai seluruh kelas dan menguji ketiga jenis relasi.
   - Mewarisi kelas turunan kedua (bukan kelas induk langsung) dan menambahkan atribut jumlah layer.
   - *Constructor* menerima nama, jumlah data latih, jumlah kelas, dan jumlah layer; gunakan *constructor* kelas di atasnya.
   - Override method: tampilkan informasi dari kelas di atasnya.
   - Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) + (jumlah layer × 200).

Jawaban :

ModelDeepLearning.java

```
package com.fembi.unguided;

public class ModelDeepLearning extends ModelKlasifikasi{
    private int jumlahLayer;

    public ModelDeepLearning(String nama, int jumlahDataLatih,
                             int jumlahKelas, int jumlahLayer) {
        super(nama, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

Output :

-

Penjelasan :

Kelas ini mewarisi ModelKlasifikasi, bukan langsung dari Model. Dengan demikian, atribut dan perilaku yang diwariskan mencakup informasi jumlah kelas. Konstruktor menggunakan super() untuk meneruskan parameter ke kelas induk.

Method tampilkanInformasi() menampilkan informasi model, jumlah data latih, jumlah kelas, dan jumlah layer. Sementara itu, hitungBiaya() menghitung biaya klasifikasi lalu menambahkan jumlah layer dikali 200.

5. Class Main
   - Buat tiga objek berikut:

Jenis model	**Nama**	**Data latih**	**Atribut tambahan**
Regresi	"Regresi Linier"	500	4 fitur
Klasifikasi	"Decision Tree"	800	3 kelas
Klasifikasi *deep learning*	"Neural Network"	1000	5 kelas, 3 layer

   - Simpan ketiga objek tersebut dalam satu array bertipe kelas induk.
   - Gunakan perulangan untuk setiap elemen array: tampilkan informasi model, tampilkan biaya komputasi, lalu jumlahkan seluruh biaya.
   - Setelah perulangan, tampilkan total biaya.
   - Terakhir, tampilkan juga:
     - biaya 10 *epoch* untuk model "Neural Network" (ambil dari array),
     - biaya 10 *epoch* untuk model "Regresi Linier",
     - biaya 10 *epoch* dengan ukuran *batch* 50 untuk model "Regresi Linier".

Jawaban :
Main.java

```
package com.fembi.unguided;

public class Main {
    public static void main(String[] args) {
        ModelRegresi regresi =
                new ModelRegresi("Regresi Linier", 500, 4);

        ModelKlasifikasi klasifikasi =
                new ModelKlasifikasi("Decision Tree", 800, 3);

        ModelDeepLearning deepLearning =
                new ModelDeepLearning("Neural Network", 1000, 5, 3);

        Model[] daftarModel = {
            regresi, klasifikasi, deepLearning
        };

        int totalBiaya = 0;

        for (Model model : daftarModel) {
            model.tampilkanInformasi();
```

```
            int biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");

            totalBiaya += biaya;
        }

        System.out.println("Total biaya: " + totalBiaya);

        System.out.println("Biaya 10 epoch (Neural Network): "
                + daftarModel[2].hitungBiaya(10));

        System.out.println("Biaya 10 epoch (Regresi Linier): "
                + daftarModel[0].hitungBiaya(10));

        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): "
                + regresi.hitungBiaya(10, 50));
    }
}
```

Output :

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Penjelasan :

Kelas Main membuat tiga objek sesuai soal, kemudian menyimpannya dalam array bertipe Model. Perulangan for-each digunakan untuk menampilkan informasi, menghitung biaya setiap model, dan menjumlahkan seluruh biaya.

Penggunaan array superclass untuk menampung objek subclass merupakan penerapan polymorphism. Method yang di-override akan dijalankan sesuai jenis objek sebenarnya. Di bagian akhir, program menghitung biaya 10 epoch untuk Neural Network, biaya 10 epoch untuk Regresi Linier, dan biaya 10 epoch dengan batch 50 untuk Regresi Linier.

D. Kesimpulan

Berdasarkan praktikum Modul 6, inheritance memungkinkan suatu kelas mewarisi atribut dan method dari kelas lain sehingga kode dapat digunakan kembali dan lebih terstruktur. Penggunaan super membantu memanggil konstruktor maupun method superclass, sedangkan overriding memungkinkan subclass menyesuaikan perilaku method sesuai kebutuhannya. Overloading memungkinkan beberapa method memiliki nama yang sama dengan parameter berbeda. Selain itu, polymorphism memungkinkan beberapa objek subclass disimpan dalam satu array bertipe superclass dan diproses melalui perulangan yang sama. Melalui program perbandingan biaya komputasi model machine learning, konsep-konsep tersebut dapat diterapkan secara langsung untuk membuat program Java yang lebih terorganisasi, fleksibel, dan mudah dikembangkan.